從程式語意到 MIPS 指令

Compiler 不必直接知道電晶體如何切換，它只需產生符合 ISA 契約的指令。Microarchitecture 不必理解 `if` 或函式，它只需正確完成 commands、位址計算、例外與記憶體效果。

本章用 MIPS32 建立完整的軟硬體介面，之後再以 ARM 與 x86 對照。學習重點是可追蹤的資料流與位址，不是背誦所有指令。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習路線與必要符號

本章使用下列符號。

- $R[x]$ 表示 index 為 x 的 32-bit register。
- $M[a]$ 表示 byte-addressable memory 中由位址 a 開始的資料。
- PC 是目前討論指令的位址；定址公式中的 $PC_{\text{next}} = PC + 4$。
- $I[k:j]$ 是 instruction I 的第 k 至第 j 個 bit。

本章採傳統 pre-Release-6 MIPS32，保留一個 branch/jump delay slot；所有控制轉移範例以 `nop` 明示延遲槽。`jal` 寫入 `$ra=PC+8`，不是 `PC+4`。本章不模擬早期 R2000 的 load delay。所有 MIPS core instructions 在案例中長度固定為 32 bits。Alignment 假設是 instruction address 為 4-byte aligned，word load/store 位址也為 4-byte aligned。

若實際 assembler 會重排指令，須使用其對應的順序控制（例如支援時使用 `.set noreorder`）並檢查 pseudo-instruction 展開，才能保證本文的 delay-slot 追蹤。

先固定 alignment 與 endianness 假設，否則同樣位址會得到不同 byte interpretation。

2.1 ISA 是軟硬體契約

ISA 描述 programmer-visible state 與 operations。MIPS32 的可見狀態包括 32 個 general-purpose registers、program counter、floating-point registers、HI/LO、status 與 exception-related registers。

程式可依 instruction semantics 依賴某些結果，但不能依賴 pipeline 有幾階或 cache 放哪裡。Timing 可以由 microarchitecture 改變，architectural result 必須維持。

ISA 也定義哪些 situations 必須產生 exception，例如 arithmetic overflow 或 address fault。精確例外的條件由 architecture 與特權規格確定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.2 MIPS 運算指令

Arithmetic instruction 採三-operand形式。例如

```
add  $t0, $t1, $t2
```

語意是

$$R[t0] \leftarrow R[t1] + R[t2].$$

`sub` 做二補數減法。`add` 與 `addi` 在 signed overflow 時依規格產生 exception，`addu` 與 `addiu` 則不因 signed overflow 產生同樣例外。選擇哪個 instruction 會改變 architecturally visible trap。

`addiu` 的名稱不代表只有 unsigned operands。它是 immediate arithmetic without signed-overflow trap，負 immediate 仍經 sign extension。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

指令與語言的表達力

ISA 的核心運算數量可以很少，compiler 用組合完成高階語言 semantics。乘法可由 iteration、shift-and-add 或乘法單元實作，但 ISA 是否提供 `mult` 會影響 instruction count。

固定長度 encoding 使 decode 位置穩定，卻限制單一 instruction 能容納的 immediate bits。複雜 instruction 可減少 instruction count，也可能拉長 critical path。

比較 ISA 時要同時看 instruction count、每 instruction work、encoding 複雜度與 compiler 能否有效使用。

不要用 RISC 或 CISC 標籤直接預測效能。第 4 章會看到 microarchitecture 可以縮小兩者差異。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.3 運算元與暫存器

MIPS32 有 32 個 32-bit general-purpose registers。C 的 scalar 變數通常先放入 registers，load/store instructions 才處理 memory。

這形成 load-store architecture。Arithmetic 指令的 operands 是 registers 或 encoded immediate，不直接指定兩個 memory locations。

Register file 的讀取埠、write port 與 bypass 路徑屬 microarchitecture。ISA 只保證寫入後的 architectural value，不保證在 pipeline 哪一 cycle 可見。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

記憶體運算元

典型 MIPS load/store 的 effective address 是

$$EA = GPR[base] + sign_extend(imm16).$$

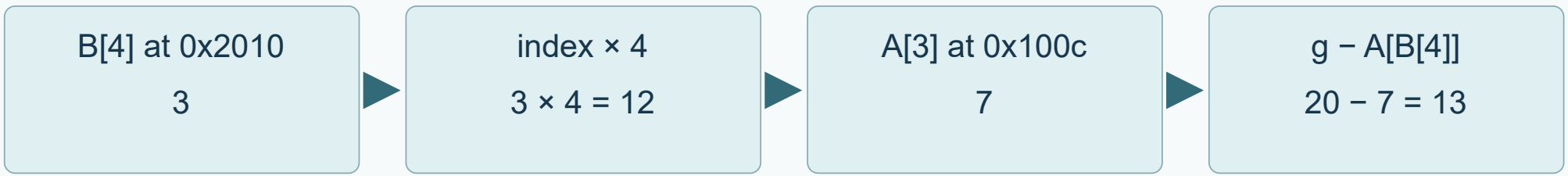
`lw $t0, 12($s0)` 把 $M[EA]$ 起算的 32-bit word 載入 register。`sw` 則把 register 值寫到 memory。

Immediate 是有號 16-bit displacement，範圍 -32768 到 32767 bytes。大型 offset 或不規則位址需先計算 base register。

若 base 是 32-bit，加法在 modulo 2^{32} 位址空間中發生。Instruction memory alignment 另行限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

間接索引：逐步追蹤兩次讀取



假設兩個陣列的 word 均為 4 bytes，base $A = 0x1000$、$B = 0x2000$，$B[4] = 3$、$A[3] = 7$、$g = 20$。所有索引、位址與讀取均有效且對齊；範例不發生算術例外。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

間接索引：逐步追蹤兩次讀取 （續）

```
lw   $t0, 16($s7)   # B[4] = 3
sll  $t0, $t0, 2    # 3 × 4 = 12 bytes
addu $t0, $t0, $s6  # address A[3] = 0x100c
lw   $s0, 0($t0)    # A[3] = 7
sub  $s0, $s1, $s0  # f = 20 − 7 = 13
```

其中 `$s6=0x1000`、`$s7=0x2000`、`$s1=20`；$B[4]$ 的 16 是 byte offset，不是元素索引本身。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

立即值

`addi $t0, $t0, -7` 把 encoded 16-bit 欄位 sign-extend 後加到 register。`lui` 把 16-bit immediate 放入 upper half，常用兩條 instruction 合成 32-bit constant。

對 $0x12345678$，可概念化為 `lui $t0, 0x1234` 後再接 `ori $t0, $t0, 0x5678`。`lui` 不用 sign extension 的方式形成常數，而是把 lower 16 bits 清零。

ISA 是否提供 single-instruction 32-bit literal 是 encoding 取捨。固定 32-bit instruction 不可能同時保留五個 register fields 與任意 32-bit immediate。

`lui` 加 `ori` 組合立即值；load 則讀取當時的記憶體，可能遭遇位址、權限或頁面例外。只有常數內容固定、位址有效且沒有額外可觀察副作用時，兩者的載入值才可相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.4 整數表示與溢位

Unsigned n-bit integer 的範圍是 0 到 $2^n - 1$。Two's complement 將最高位權重設為 -2^{n-1}，範圍是

$$-2^{n-1} \le x \le 2^{n-1} - 1.$$

對 32 bits，範圍是 -2147483648 到 2147483647。負數的 bit pattern 可由 complement 加 1 得到。

Signed operation 與 unsigned operation 可使用相同加法器，差別在 overflow interpretation、comparison 與乘除法處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

溢位與比較

Signed addition overflow 不是 carry-out 的別名。兩個同號 operands 相加得到異號結果時，結果越界。

溢位判斷可用 sign bits。令 operands 的 sign 為 s_a, s_b，result 為 s_r：

$$overflow = \overline{s_a \oplus s_b} \wedge (s_a \oplus s_r).$$

Unsigned addition 的 overflow 看 carry-out。比較也有 signed 與 unsigned 版本，所以 MIPS 提供 `slt` 與 `sltu`。

`0xffffffff + 1` 在 unsigned 是 overflow，在 two's complement 是 0 且無 signed overflow。必須先指定 interpretation。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

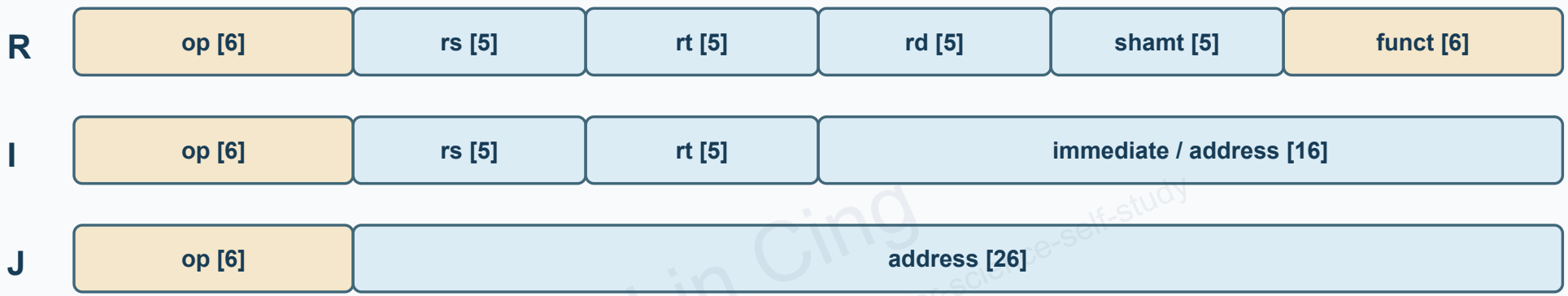
2.5 MIPS 指令格式

MIPS core instruction 一律 32 bits。R-type 用 opcode 000000 加 `funct` 區分 ALU operation；I-type 用 opcode 選擇 operation 與 16-bit immediate；算術及位址 displacement 做 sign extension，邏輯立即值做 zero extension；J-type 提供 26-bit jump index。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.5 MIPS 指令格式 （續）

MIPS R、I、J instruction 欄位



所有欄位總和固定 32 bits；I 的 sign/zero extension 與 store data 來源依 opcode 解讀。

固定欄位位置使 fetch 後可並行讀取 opcode、registers 與 immediate。Encoding 的精簡會把更多 decision 推到 compiler 與多 instruction sequence。

欄位圖是概念示意，不代表實際晶片佈局。Bit order 從 MSB 31 往 LSB 0。

R、I、J 三種格式

R-type 的 MSB 到 LSB 欄位是 `op[31:26]`、`rs[25:21]`、`rt[20:16]`、`rd[15:11]`、`shamt[10:6]`、`funct[5:0]`。

I-type 是 `op`、`rs`、`rt`、`imm16`。Branch 使用 immediate 計算 offset；load/store 使用 immediate 計算有效位址；store data 來自 `rt`，不是 immediate。

J-type 是 `op` 與 `target26`。它不能任意指定 32-bit absolute address，因此要和程式所在 region 的 upper PC bits 合併。

I-type 的 `rt` 在 `addi` 常作 destination，在 store 作 source。欄位名稱相同，角色由 opcode 決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.6 邏輯與移位

Bitwise operations 是 `and`、`or`、`nor` 與 `xor`。Immediate forms 用 zero-extended 16-bit immediate，因為遮罩通常是 bit pattern。

Logical shift 補 zero。Arithmetic right shift 補 sign bit。MIPS 以 `srl`、`sra`、`sll` 操作 registers，shift amount 可為 `shamt`。

例如要取出 word 的 bits 11 到 8：

```
srl $t1, $t0, 8
andi $t1, $t1, 0x000f
```

`and` 常用於清零不需要的 bit；`or` 用於設定 bit；`xor` 可切換 bit。Mask 必須配合 shift amount。

2.7 條件分支與迴圈

`beq rs, rt, label` 在 equality 成立時改變 control flow。若不成立，architecture-visible next PC 是 sequential instruction。

本章的傳統 MIPS32 模型有一個 branch delay slot，緊接 branch 的指令無論分支是否成立都會執行。例子以 `nop` 避免把不安全操作放入延遲槽；Release 6 的無延遲槽控制轉移不在本章模型內。

Branch 型態通常包括 equal、not equal、less-than 等比較。組合語言 pseudo-instruction 可由 branch 與 `slt` 展開。

必須宣告使用 classic MIPS 還是 MIPS32 release 的 ISA。不同版本對 branch delay、pseudo-instructions 與 instruction availability 可能不同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

將迴圈翻成指令

C 片段：

```
int sum = 0;
for (int i = 0; i < n; ++i)
    sum += a[i];
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

將迴圈翻成指令 （續）

令 `$a0` 指向 `a`、`$a1` 為 `n`、`$v0` 為 sum：

```
move $v0, $zero
move $t0, $zero
beq  $t0, $a1, done
nop
loop:
sll  $t1, $t0, 2
addu $t1, $t1, $a0
lw   $t2, 0($t1)
addu $v0, $v0, $t2
addiu $t0, $t0, 1
bne  $t0, $a1, loop
nop
done:
```

將迴圈翻成指令 （續）

前提是signed `n >= 0`、array位址有效且不wrap，並限定signed C sum不溢位，才與上面C語意等價；MIPS `addu`本身仍採modulo 2^32。`sll` 將 index 乘 4，因為 word 是 4 bytes。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Switch 的策略

密而集中的 case labels 可用 jump table。先檢查 index 是否在 table 範圍，再以 `sll` 與 `lw` 取得 target address，最後 `jr`。

稀疏或數目很少的 labels 可編成 compare-and-branch chain。Jump table 常降低決策鏈長度，但增加 table 空間、indirect jump 與 cache footprint。

編譯器還可能反向排列 cases、用 binary search tree 或 range checks。策略依 label distribution、安全性與 microarchitecture 選擇。

Direct branch的target由PC與immediate固定；jump table的同一個indirect-jump PC可隨index指向多個targets，需預測的不只有taken方向。Table若不在cache，還會先等target-address load；因此決策鏈較短不保證elapsed time較短。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從 bit field 到 control flow 的完整追蹤

解碼不能只查 mnemonic。以下三個 word 分別覆蓋 I、R 與 J 格式的關鍵差異。

32-bit word	欄位拆解	執行語意
`0x8fa8000c`	`op=0x23`、`rs=29`、`rt=8`、`imm16=12`	`lw $t0, 12($sp)`；`EA=R[sp]+12`
`0x012a4020`	`op=0`、`rs=9`、`rt=10`、`rd=8`、`shamt=0`、`funct=0x20`	`add $t0,$t1,$t2`
`0x0c100140`	`op=3`、`target26=0x100140`	`jal` 至同一個 256-MiB region 的 `0x00400500`

I-format 的 `imm16` 並非一律 signed。`lw` 與 `addi/addiu` 做 sign extension；`andi/ori/xori` 做 zero extension；store 的 `rt` 是資料來源。opcode 先決定格式與欄位角色，不能先假定 `rt` 一定是 destination。

ISA 結果與高階語言語意也要分開。MIPS `addu/addiu` 的 32-bit result 是 modulo 2^{32}；C 的 unsigned arithmetic 也是 modulo 2^N，但 signed overflow 在 C 標準中屬於 undefined behavior。Compiler 不能只因兩個 operands 是 signed 就把任意越界程式翻譯成有保證的 modulo 結果；原始程式需先證明不越界，或改用 unsigned／更寬型別。

令 `beq` word 是 `0x1109fffc`，且 instruction 位於 `0x00400020`。`imm16=0xfffc=-4`，因此

從 bit field 到 control flow 的完整追蹤 (續)

$$PC_{\text{next}} = 0x00400020 + 4 = 0x00400024,$$

$$target = PC_{\text{next}} + (\text{sign_extend}(0xfffc) \ll 2) = 0x00400024 - 16 = 0x00400014.$$

在 classic MIPS32 的一個 delay slot 模型下，branch taken 的 execution sequence 是 `0x00400020 → 0x00400024 → 0x00400014`；not taken 是 `0x00400020 → 0x00400024 → 0x00400028`。`0x00400024` 無論條件為何都執行，因此不能把它誤當成只在 taken 時執行。

對 `jal` 位於 `0x00400070` 的例子，delay slot 在 `0x00400074`，target 是 `0x00400500`，`$ra=0x00400070+8=0x00400078`。Return sequence `jr $ra; nop` 會先執行 return 的 delay slot，才到 `0x00400078`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從 bit field 到 control flow 的完整追蹤 （續）

控制轉移的 PC、delay slot 與 return address



Branch offset 以 instruction 計；jal 寫回的 ra 越過 delay slot。

Return 仍要執行 jr 的 delay slot；taken/not-taken 只改變 branch 後的 target。

三個必要邊界是 instruction address 必須 word aligned、branch offset 以 instruction 為單位、J-type target 只能沿用 PC+4 的高四位。任一把 byte offset、word offset 或目前 PC 的高四位混用，換一組輸入就會得到錯的 target。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

2.8 程序呼叫契約

`jal target` 把 return address 放入 `$ra`，再跳到 target。本章傳統 MIPS32 明定 `$ra = PC+8`，返回位置越過 call 的一個 delay-slot instruction。

`jr $ra` 跳回 caller。被呼叫程序若要呼叫其他程序，必須先保存 caller 的 `$ra`，否則 nested call 會覆寫。

Calling convention 是 ABI 的一部分，不是 processor 硬體自動完成的全部工作。Register allocation、stack layout 與保存順序由規格約定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

暫存器分工

教材案例中的 MIPS convention 通常是：

- `$zero` 固定為 0。
- `$a0` 至 `$a3` 傳 arguments。
- `$v0`、`$v1` 傳 return values。
- `$t0` 至 `$t9` 由 caller 保存。
- `$s0` 至 `$s7` 由 callee 保存。
- `$sp` 是 stack pointer，`$ra` 是 return address。

Caller-saved 的意思是 caller 若要在 call 後繼續依賴該 register，就要自己保存。Callee-saved 則表示 callee 有義務在 return 前恢復原值。

這些名稱是 convention 與 ABI 的責任，硬體不會依 `$t` 或 `$s` 名稱自動保存 register。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

巢狀呼叫與 $ra

假設 `main` 呼叫 `f`，`f` 再呼叫 `g`。`jal g` 會覆寫 `f` return 到 `main` 所需的 `$ra`。

因此 `f` 在第一次 call 之前保存 `$ra` 到自己的 frame，等 `g` 返回後再 restore。

以下8-byte frame只示範return-address保存，不宣稱完整o32 outgoing-argument配置。最小sequence可概念化為：

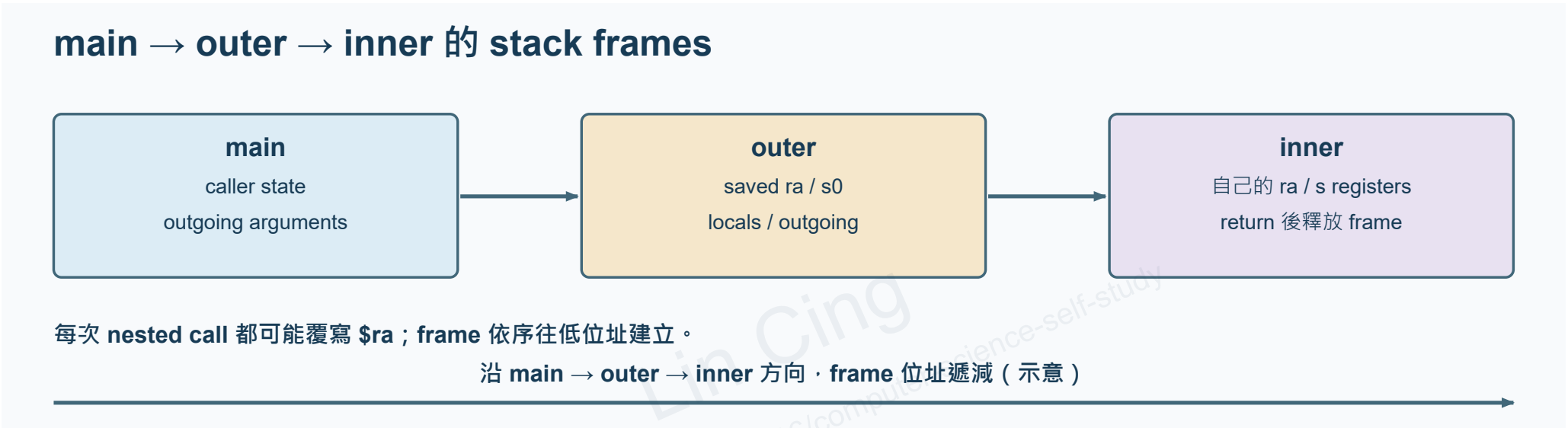
```
addiu $sp, $sp, -8
sw    $ra, 4($sp)
jal   g
nop
lw    $ra, 4($sp)
addiu $sp, $sp, 8
jr    $ra
nop
```

巢狀呼叫與 $ra （續）

Frame size 保持 alignment 是 ABI 要求，本例以 8 bytes 為例。實際 frame 還要保存 arguments、locals 與 callee-saved registers。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

堆疊框



Call frame 通常包含 saved `$ra`、saved `$s` registers、local arrays、spill registers 與 outgoing argument area。Frame size必須容納同時存活的slots，offset需落在frame內且滿足各slot alignment，不能以offset大小代替所需空間。

Stack pointer 在 entry 減去 frame size，在 exit 前恢復。每個 stack slot 的 alignment 與 argument area 由 ABI 決定。

堆疊框 （續）

若 pointer 指向已釋放 frame，後面 call 可能覆寫它。Lifetime 錯誤是語意問題，編譯器未必能自動修正。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Heap 記憶體

Stack 隨 procedure lifetime 成長與縮回。Heap 用 `malloc` 或 `new` 配置，lifetime 由 program 控制，直到 `free` 或 garbage collection。

Allocator 管理 free blocks、metadata 與 fragmentation。Allocation 可能失敗，也可能觸發 page fault 與作業系統擴張 virtual address space。

Heap 的 logical allocation 與實際 physical residency 不同。成功取得 virtual memory 不代表每頁已載入 DRAM。

若函式回傳指向local int的pointer，return後該物件生命期已結束，後續dereference不是合法存取；stack bytes暫未覆寫也不延長其生命期。Heap leak則是在配置後失去最後一個可釋放的pointer，例如反覆malloc再覆寫唯一pointer；物件仍占配置空間，與dangling pointer的「存取已失效物件」不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.9 字元、字串與位元組

ASCII 定義 7-bit 碼位，通常存於一個 8-bit byte；Unicode 則透過 UTF-8、UTF-16 等編碼表示。C 字串以終止 byte 表示長度；其他語言可能同時保存 length metadata。

MIPS 以 `lb`、`lbu`、`sb` 存取 byte。`lb` 做 sign extension，`lbu` 做 zero extension。String copy 要維持 destination capacity、alignment policy 與終止符。

Byte order 會影響 multi-byte word 在 memory 中的排列。Endianness 必須在 data exchange 與 binary format 邊界明確。

同一組 bytes `41 42 43 00` 作 NUL-terminated ASCII string 是 "ABC"；作32-bit integer，big-endian 是 `0x41424300`，little-endian 是 `0x00434241`。讀到同樣 bits 並未決定其型別語意。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.10 大型立即值

Fix-width instruction 的 immediate field 有限。要把 arbitrary 32-bit constant 放入 register，需要多 instruction。

`lui rt, imm16` 將 `imm16` 放到 bits 31 至 16，low half 清零。接著 `ori rt, rt, imm16` 填 low half。

若 compiler 能在 symbol relocation 時直接填入 address，`lui` 與 `addiu` 也可能形成 address sequence。Effective address 最終仍是 32-bit modulo address space。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分支與跳躍定址

PC-relative branch 的 target 為

$$target = (PC_{\text{next}}) + (sign_extend(imm16) \ll 2).$$

Offset 的單位是 instructions，不是 bytes。16-bit signed offset 可表示的 byte 範圍約為 $\pm 2^{17}$，且 instruction target 必須 4-byte aligned。

J-type 的 target 為

$$target = (PC_{\text{next}}[31:28] \parallel target26 \parallel 00_2).$$

它保留 PC+4 的高四位，所以目標限於 PC+4 所屬的 256-MiB region；當指令位於 region 末尾時，不應直接改用目前 PC 的高四位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

定址模式總覽

教材包含 register、immediate、base-plus-displacement 與 PC-relative addressing。`lw $t0, 0($sp)` 使用執行該指令時的 architectural base value 加 displacement；資料相依由實作的前送或停頓處理，branch delay slot 並不是保存 base 的機制。

Addressing mode 不是越多越好。每增加一個模式，就增加 decode、exception priority 與編碼複雜度。

Pseudo-direct jump 用小 target field 換取固定 encoding，代價是 region 限制。Compiler 與 linker 若把 target 放錯 region，可能需要 trampoline 或 indirect jump。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

解碼機器碼

以 `0x012a4020` 為例，依序拆欄位：

```
op      = 000000
rs      = 01001  ($t1)
rt      = 01010  ($t2)
rd      = 01000  ($t0)
shamt   = 00000
funct   = 100000 (add)
```

所以它是 `add $t0, $t1, $t2`。decoder 先看 opcode，再依 format 選 `funct` 或 immediate。

解碼必須先確認 ISA、register naming 與 instruction length。若把 x86 variable-length bytes 當成 MIPS 固定欄位，會得到沒有意義的結果。

2.11 同步與原子操作

多個 cores 共享 memory 時，單純 `lw`、`addiu`、`sw` 會形成 read-modify-write race。兩個 cores 可能都讀到舊值，再各自寫入新值。

MIPS 的 `ll` 與 `sc` 提供 atomic pair。`ll` 讀取並建立 reservation；`sc rt, offset(base)` 以 `rt` 的舊值作 store data，成功則原子寫入並把 `rt` 改為 1，失敗則不寫入並把 `rt` 改為 0。

失敗時常見策略是 retry loop：

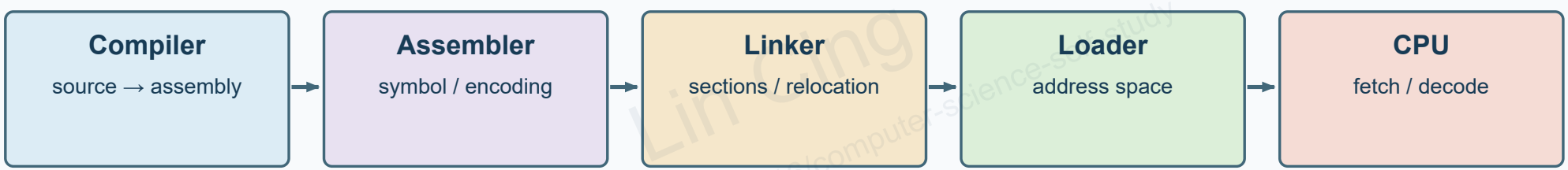
```
loop:
ll   $t0, 0($s0)
addiu $t0, $t0, 1
sc   $t0, 0($s0)
beq  $t0, $zero, loop
nop
```

具體 retry 次數與 fairness 不是 ISA 保證。若 reservation 被另一個 store 或例外中斷，`sc` 可能失敗。

2.12 從 C 到執行檔

Compiler 做型別檢查、最佳化與 instruction selection。Assembler 把 symbols 與 pseudo-instructions 轉成 machine instructions 與 relocation records。Linker 合併 object files、解析 external references、配置 sections。Loader 建立 process address space 與初始 registers。

C、assembly、object、executable 與 runtime 工具鏈



Relocation 在 object 中保存 type 與 addend，linker 才填 final displacement 或 address。

source line、ISA instruction、execution cycle 是三個不同計數，不能互相替代。

每個階段都可能失敗。Compiler 的 undefined behavior、assembler 的 invalid operand、linker 的 undefined symbol、loader 的 permission 或 mapping 問題必須分開診斷。

2.12 從 C 到執行檔 （續）

以外部 `printf` 為例，object 留下 undefined symbol 與 call relocation；靜態連結器產生可執行檔；選擇動態連結時，可依 ABI 建立 stub、間接表或重定位，再由 dynamic loader 映入提供者並解析 runtime address。完全靜態連結則不需要動態載入器解析該符號。若採 lazy binding，首次經 resolver 更新表項，後續走已解析的間接路徑。實際 stub 與暫存器規則依 ABI，不能把此跨階段流程當作固定 MIPS 指令序列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

工具鏈的地址語意

Assembler 第一次看到 local label 時可計算 section-relative address，但 external symbol 尚未確定。Object file 保存 relocation type 與 addend，讓 linker 填入最終 displacement 或 address。

Static linking 在 executable 內解析 references。Dynamic shared library 可能用 position-independent code 與 runtime resolver，降低 multi-process physical memory duplication。

Debug 時要分清楚 virtual address、file offset、load segment offset 與 physical page。它們不是同一個數字。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

呼叫慣例案例

```
int twice_sum(int x, int y) {
    int s = x + y;
    return s * 2;
}
```

假設 signed 加法與乘 2 的 mathematical result 都可表示，概念序列如下：

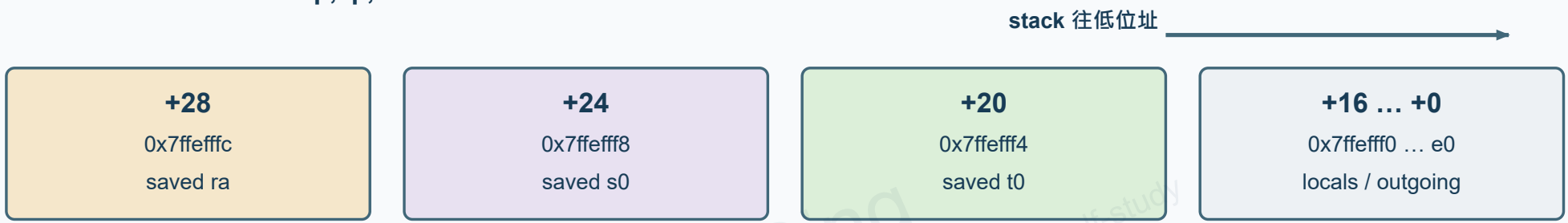
```
twice_sum:
addu $v0, $a0, $a1
sll  $v0, $v0, 1
jr   $ra
nop
```

這個 leaf function 沒有 call，不必保存 `$ra`；它只寫 `$v0`，也不破壞 caller 依賴的 `$s` registers。

固定 frame 的 slot 位址

固定 frame：slot 位址與釋放邊界

SP 0x7fff0000 → addiu sp,sp,-32 → 0x7ffeffe0

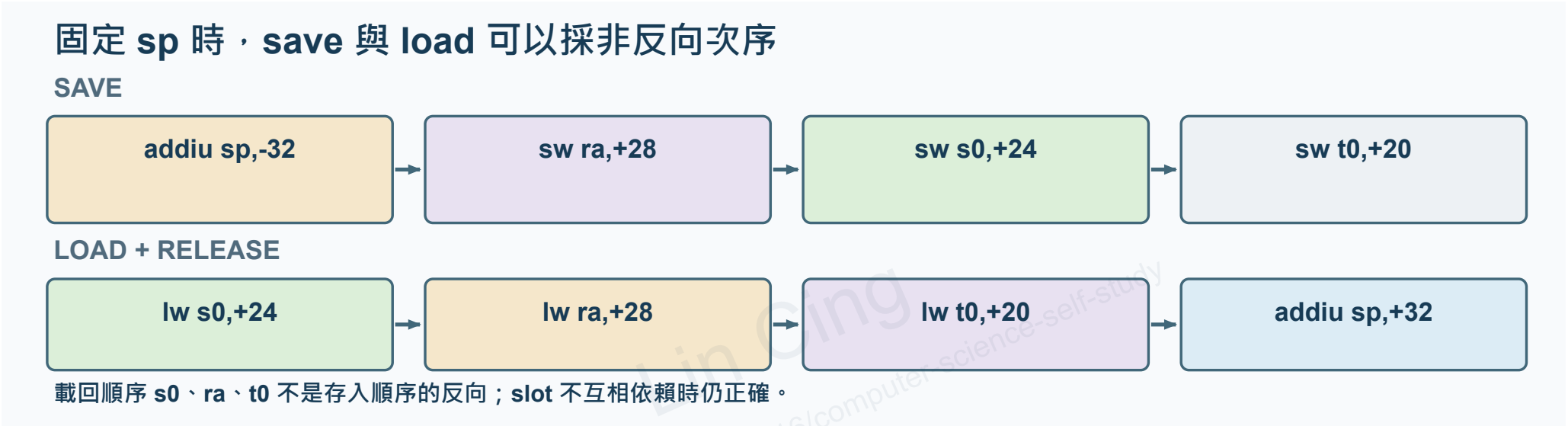


frame 上限不屬於本 frame；低端 sp 是本 frame 的起點。

位址固定是「可依 offset 任意配對」的前提，但 sp 仍必須等所有 values 載回後才釋放。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

固定 frame 的 save、load 與 release 時序



令進入函式前 `sp=0x7fff0000`，frame 為 32 bytes。`addiu sp,sp,-32` 後 `sp=0x7ffeffe0`，三個獨立 slot 的位址固定如下。

固定 frame 的 save、load 與 release 時序 (續)

Slot	位址	存入	載回
`28(sp)`	`0x7fefffc`	`ra`	`ra`
`24(sp)`	`0x7fefff8`	`s0`	`s0`
`20(sp)`	`0x7fefff4`	`t0`	`t0`

一個有效序列是依序存 `ra`、`s0`、`t0`，再依序載回 `s0`、`ra`、`t0`，最後才 `addiu sp,sp,32`。載回次序並非存入次序的反向，結果仍正確，因為 `sp` 在載回期間不變，三個 offset 指向不同 slots，且 load destinations 不互相依賴。

「save 與 restore 必須反向」只適用於 push/pop 會改變 `sp` 的 LIFO 序列，或 slot/register 之間存在依賴。固定 frame 的釋放時機則是硬性條件：所有需要的 values 都載回後，才可增加 `sp`；先用新的 `sp` 讀取舊 frame 會錯位。

遞迴呼叫的 frame trace

令 `main` 在 `0x00400010` 執行 `jal f`，`f` 的回傳位址是 `0x00400018`。`f` 進入時先令 `$sp=0x7fff0000`，配置 32-byte frame 後 `$sp=0x7ffeffe0`：

寫入	位址	值
`sw $ra,28($sp)`	`0x7ffefffc`	`0x00400018`
`sw $s0,24($sp)`	`0x7ffefff8`	caller 的 `$s0`

`f` 在 `0x00400120` 再執行 `jal f`，新的 `$ra` 是 `0x00400128`。inner activation 令 `$sp=0x7ffeffc0`，並把 `0x00400128` 存到 `0x7ffeffdc`。此時兩個 frames 同時存活，任何 `$sp+28` 都指向目前 activation 的 slot，不能拿 inner 的 `$sp` 讀 outer frame。

Base case 返回時，inner 依序載回自己的 `$ra` 與 `$s0`，把 `$sp` 加回 `0x7ffeffe0`，再 `jr` 到 `0x00400128`。Outer activation 繼續執行，最後從 `0x7ffefffc` 載回 `0x00400018`、釋放 32 bytes，再回到 `main`。每個 `jal` 與 `jr` 後都保留 `nop` 作為本章模型的 delay slot；移除它會改變實際執行的 instruction。

遞迴呼叫的 frame trace （續）

這個 trace 同時驗證三件事。第一，`$ra` 必須在 nested call 前保存；第二，frame 的 slot 位址由目前 `$sp` 與固定 offset 決定；第三，release 必須等所有需要的 loads 完成後才增加 `$sp`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

陣列與指標的指令差異

Array indexing 的位址通常是

$$address = base + i \times element_size.$$

Pointer traversal 每次增加 `element_size`，迴圈中少一次 index scaling，但 compiler 可能在 array 版本做相同 strength reduction。

指標本身可以是 64-bit，也可以指向 invalid 或 one-past-end 之外的位址。C/C++ 的 pointer arithmetic 有 language-level 邊界，超過有效物件範圍即使 machine 能形成 bit pattern，也不保證合法。

索引版本每次可包含 `sll index,2` 與 base addition；pointer版本把位址保存在暫存器、每次加4。下方 clear 範例將兩種穩態序列完整展開為7與5個指令位置；此差距包含 loop test 與 delay slot，但不含 prologue、exit 或 memory stalls。Compiler strength reduction 可把前者改成後者。

C 排序的完整資料流與成本

以下用插入式相鄰交換排序，展示 C 控制流、陣列定址與程序呼叫：

```
for (int i = 0; i < n; i += 1)
  for (int j = i - 1; j >= 0 && v[j] > v[j + 1]; j -= 1)
    swap(v, j);
```

取 `v=[3,1,2]`、`n=3`。`i=0` 時 inner loop 不安裝；`i=1` 時 `j=0`，因 `v[0]=3>v[1]=1` 呼叫 `swap(v,0)`，陣列變成 `[1,3,2]`；`i=2` 時先比較 `v[1]=3>v[2]=2`，交換成 `[1,2,3]`，再比較 `v[0]=1≤v[1]=2` 而停止。此時 sorted invariant 成立。

以下把完整控制流寫成 classic MIPS32；每個 branch 與 `jal` 後的 `nop` 都是本章模型的 delay slot。 此處sort與所附leaf swap採精簡教學呼叫契約；24-byte frame不保留o32要求的16-byte outgoing argument home area，因此只保證與本例leaf callee搭配，不能直接與任意o32編譯的callee互換。若要求完整o32互通，需另為outgoing area保留0..15 offsets並移動saved slots。前置條件是 `n>=0`、`v` 指向 `n` 個 aligned words，且 index calculation 不讓有效位址越界。

C 排序的完整資料流與成本 （續）

```
sort:
addiu $sp, $sp, -24
sw    $ra, 20($sp)
sw    $s3, 16($sp)
sw    $s2, 12($sp)
sw    $s1, 8($sp)
sw    $s0, 4($sp)
addu  $s2, $a0, $zero
addu  $s3, $a1, $zero
addu  $s0, $zero, $zero
for1:
slt   $t0, $s0, $s3
```

https://gi

C 排序的完整資料流與成本 （續）

```
beq   $t0, $zero, exit1
nop
addiu $s1, $s0, -1
for2:
slti  $t0, $s1, 0
bne   $t0, $zero, exit2
nop
sll   $t1, $s1, 2
addu  $t2, $s2, $t1
lw    $t3, 0($t2)
lw    $t4, 4($t2)
slt   $t0, $t4, $t3
```

https://gi

C 排序的完整資料流與成本 （續）

```
beq   $t0, $zero, exit2
nop
addu  $a0, $s2, $zero
addu  $a1, $s1, $zero
jal   swap
nop
addiu $s1, $s1, -1
j     for2
nop
exit2:
addiu $s0, $s0, 1
j     for1
```

https://gi

C 排序的完整資料流與成本 （續）

```
nop
exit1:
lw    $s0, 4($sp)
lw    $s1, 8($sp)
lw    $s2, 12($sp)
lw    $s3, 16($sp)
lw    $ra, 20($sp)
addiu $sp, $sp, 24
jr    $ra
nop
```

令 `v=0x10010000`、`k=1`。`swap` 先算 `k*4=4`，兩元素位址是 `0x10010004` 與 `0x10010008`；它載入兩個 word，再以一個暫存 register 完成 store。Leaf procedure 只使用 caller-saved temporaries，不改用 `$ra` 以外的 saved state，因此不必配置 frame，但 `jr` 在本章 classic MIPS 模型仍需要 delay slot：

C 排序的完整資料流與成本 （續）

```
swap:
sll  $t1, $a1, 2
addu $t1, $t1, $a0
lw   $t0, 0($t1)
lw   $t2, 4($t1)
sw   $t2, 0($t1)
sw   $t0, 4($t1)
jr   $ra
nop
```

只用一個迴圈 iteration 的穩態成本比較 array index 與 pointer clear。Array 版是 `sll+addu+sw+addiu+slt+bne+nop` 共 7 個指令位置；pointer 版是 `sw+addiu+slt+bne+nop` 共 5 個。省下的 2 個是 index scaling 與 base addition，不包括 prologue、初始 guard 或 exit。現代 compiler 可用 strength reduction 與 induction-variable elimination 把 index 版轉成 pointer-like code，所以不能從原始碼寫法直接斷言效能。

C 排序的完整資料流與成本 （續）

這個成本模型要求兩版使用相同 word size、相同 memory traffic、沒有 page fault 或 trap、branch delay slot 固定存在，且比較範圍只含穩態迴圈。對 `n=100` 的指定模型，array body 是 `7×100=700` 個指令位置，optimized pointer body 是 `5×100=500` 個；兩者的初始化與最後 exit check 另計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

編譯與解譯路徑

C compiler 通常先產生 target-specific assembly。Java bytecode 不直接等於 MIPS machine code，JVM 依平台實作可 interpretation、JIT compile 或二者混合。

Interpretation 每次執行 bytecode 都付出 dispatch overhead。JIT 累積 profile 後生成 native code，改善 hot path；warm-up 與 code generation 會增加 startup latency。

ISA 相似不代表語意相同。Integer overflow、unaligned access、atomic operations 與 memory model 都可能不同。

比較 C 與 Java 時要把 type semantics、runtime 與 engine 分開，不能只看一道算術 instruction。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ARM 比較

ARM 是 load/store RISC 家族，透過 conditional execution、operand shifts 與多種 addressing 設計減少 branches。不同 ARM architecture version 的功能差異很大，不能把一個版本的 instruction 套用到全部 ARM 裝置。

與 MIPS 相同的是 registers、load/store 邊界與 procedure convention 的重要性。不同之處在 encoding、conditional flags 與 ISA extensions。

本章只讀教材版本的 ARM 案例，不把它當成所有現代 ARM64 的規格。

指出 chapter 的 ARM 例子接近 32-bit ARM 時代，與 AArch64 不可混用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

x86 比較

x86 採 variable-length instructions，encoding 可包含 prefixes、ModR/M、SIB 與 displacement。相同 instruction 可由一至多 bytes 表示。

有利於 code density 與 backward compatibility，代價是 fetch/decode alignment 複雜、instruction boundary 不一定可在固定位置找到。

Intel 與 AMD 實作會把 x86 instructions 解成 internal micro-operations，再用 superscalar pipeline 執行。ISA instruction 數不等於 microarchitecture 的 µops 數。

解釋為什麼 micro-op cache 可隱藏 variable-length decode 的成本，但不能讓 ISA 變成 MIPS。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

x86-64 的 LEA、struct layout 與可變指令長度

採 System V AMD64 ABI 的預設對齊、沒有 packing，且存取的 row 存在、$0 \le j < 4$。考慮 `struct row { int64_t id; int32_t vals[4]; int64_t sum; };`。`id` 在 offset 0，`vals` 在 offset 8、佔 16 bytes，`sum` 在 offset 24，整個 struct 對齊 8 bytes，因此 `sizeof(struct row)=32`。

對 `rows[i].vals[j]`，byte address 是

$$address = base + 32i + 8 + 4j.$$

若 `base=0x3000`、`i=5`、`j=3`，地址是 `0x3000+160+8+12=0x30b4`。假設 `i` 已擴成64-bit並放在 `%rdi`、有號 `int j` 在 `%esi`，而 `base` 在 `%rdx`，可先 sign-extend `j`、把 `i` 乘 32，再使用 SIB 的 scale 4：

```
movslq %esi, %rsi
salq   $5, %rdi
leaq   8(%rdi,%rsi,4), %rax
leaq   (%rdx,%rax), %rax
```

x86-64 的 LEA、struct layout 與可變指令長度 （續）

`lea` 只計算 address，不讀 memory；`movslq` 則把 32-bit signed index 擴成 64-bit address arithmetic。x86 instruction 是 variable length，branch displacement 以 byte 計，instruction boundary 也不固定在相同 modulo；這些規則與 fixed-width MIPS 不可互換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ISA 常見誤解

第一個誤解是「more powerful instruction 一定更快」。一個 instruction 可能只用 decoder 較少精力，但執行 latency 很長，或阻擋 parallel issue。

第二個誤解是「register 多，編譯器一定用得更好」。Register pressure、calling convention 與 spilling 仍決定實際效果。

第三個誤解是「pseudo-instruction 是硬體指令」。Assembler expansion 的 sequence 可能因 register 或 immediate 條件不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ISA 設計小結

良好 ISA 要在 encoding、compiler、例外、virtual memory、同步與 backward compatibility 之間取捨。固定格式降低 decode 成本，但需要 multi-instruction constants；variable format 增加密度，代價是複雜 decode。

Program correctness 要求 architecture-visible result 一致。Performance 則依 instruction mix、CPI、microarchitecture 與 memory path。

本章所有 address 與 condition 都要交代位元寬、alignment 與 sign/zero extension。

本章lw/sw的base及displacement以bytes計；branch immediate則以4-byte instruction為單位。ADDIU會sign-extend而ORI會zero-extend；相同16-bit欄位必須先依opcode決定解釋。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

指令集歷史

早期處理器為少量、固定欄位與簡單指令，便於實現與 compiler 產生程式。之後 CISC 加入複雜 addressing 與較高 code density；RISC 把操作規則化，將複雜工作交給 compiler。

現代處理器界線模糊。RISC instruction 可以 decode 成 μops，CISC instruction 也可以在內部拆成規則化操作。

歷史閱讀重點是設計限制，而不是把某個時代的口號當成永恆定理。

面向	機制與必須一併檢查的成本
Code size	密集encoding可減fetch bytes；複雜指令仍可能需多個內部operations
Decode	固定欄位可平行讀取；變長encoding需先辨識邊界
Memory traffic	Load/store數、寬度、locality與cache miss決定bytes，不能只看mnemonic
Parallel issue	真資料相依及functional-unit可用性限制重疊；單條做更多工作未必更易並行
Compatibility	保留舊binary需維持ISA/ABI語意；內部decode與execution可重新實作

本章重點

- ISA 版本、delay slot 與 assembler 指令順序決定控制流程。
- 位址由 base 加 byte offset；載入前先檢查元素大小、範圍與對齊。
- signed overflow 會否 trap 取決於指令；ABI 規定保存義務與 layout。
- LL/SC 的成功、記憶體排序與公平性是不同保證。

參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 2. CSO course slides, Chapter 2.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study